Pulse: AI-Powered Incident Monitoring Platform

1. Purpose of this document

This document is the starting point for building Pulse the way a product team would. Phase 0 sets up everything needed before any requirement is written: goals, tooling, repository, workflow and schedule. Phase 1 then defines what Pulse must do: the problem, users, scope, user stories, functional and non-functional requirements, and risks.

Pulse is a microservices platform that ingests logs and metrics from other services, detects error spikes, groups them into incidents, and uses an LLM to explain each incident and suggest likely causes. A React dashboard shows service health and incidents.

Document status: all targets and numbers below are proposals. Anything marked *(target)* is validated or revised during load testing in Phase 5.

2. Phase 0: Project foundation

2.1 Vision and goals

Vision: help small engineering teams understand production problems in minutes instead of hours.

Project goals

1. Build a working, demo-able system that covers frontend, backend, database, microservices, rate limiting, caching and AI API integration.
2. Follow a real SDLC with written artifacts: requirements, design, ADRs, test plan, CI/CD and a release.
3. Produce measurable results (throughput, latency, cache hit rate) that can be quoted honestly on a resume.
4. Learn the new pieces of the stack: Redis, a message broker, Lua scripting, resilience patterns and load testing.

2.2 Success criteria

Area	Success looks like
Functionality	A user can register a service, send logs, see an incident created, and read an AI summary on the dashboard
Engineering quality	Automated tests run in CI on every pull request; the system starts with one command
Performance	Load test results recorded with real numbers in the README
Documentation	README with architecture diagram, ADRs, API docs and a demo GIF
Career value	Project is on the resume with 2 to 3 bullets backed by measured metrics

2.3 Technology decisions (proposed)

Each choice gets a short ADR in Phase 2. This table is the starting position.

Concern	Choice	Reason
Language and framework	Java 21, Spring Boot 3	Existing strength and target-role relevance
API gateway	Spring Cloud Gateway	Fits the Spring ecosystem; hosts auth and rate limiting
Database	MySQL 8 with Flyway migrations	Existing experience; versioned schema
Cache and rate limiter store	Redis	Atomic operations and Lua scripts for the token bucket; fast cache
Messaging	RabbitMQ	Simple to run locally; enough for the event flow
AI integration	An LLM API called over HTTPS	Provider selected in an ADR; wrapped behind an interface so it can be swapped
Resilience	Resilience4j	Timeout, retry and circuit breaker around the AI call
Frontend	React with Vite	Existing experience
Testing	JUnit 5, Mockito, Testcontainers, k6	Unit, integration and load testing
Observability	Spring Actuator, Prometheus, Grafana	Metrics for Pulse's own services
Packaging and CI	Docker, Docker Compose, GitHub Actions	One-command startup; automated pipeline

2.4 Development environment checklist

- JDK 21 installed
- Maven installed
- Node.js LTS for the React app
- Docker Desktop (or Docker Engine) running
- IDE set up (IntelliJ IDEA/ VS Code with Java extensions)
- Git configured with name, email and SSH key
- API testing tool (Postman)
- k6 installed for load tests
- An LLM API key stored in a local environment file that is never committed
- A diagramming tool (draw.io or Mermaid) for architecture diagrams

2.5 Repository and Git workflow

Repository style: one monorepo, `pulse`, with a folder per service. A monorepo keeps shared code, docs and CI in one place, which suits a solo project.

```
pulse/
  docs/ ------------ requirements, design, ADRs, test plan
  services/
    gateway/
    auth-service/
    ingestion-service/
    analysis-service/
    ai-insight-service/
    notification-service/
  common/            shared DTOs and event classes
  frontend/          React app
  deploy/            docker-compose, monitoring config
  load-tests/        k6 scripts
  .github/workflows/ CI pipelines
```

Branching: trunk-based with short-lived feature branches.

- `main` is always working and protected (changes only through pull requests).
- Branch names: `feature/<issue-number>-short-name`, `fix/<issue-number>-short-name`.
- Every pull request links to an issue, passes CI, and is squash-merged.

Commit messages: Conventional Commits (`feat:`, `fix:`, `docs:`, `test:`, `chore:`).

Secrets: never committed. Use environment variables and a `.env.example` file with placeholder values.

2.6 Project management

- **Method:** Agile with one-week sprints.
- **Board:** GitHub Projects with columns Backlog, Ready, In Progress, In Review, Done.
- **Issues:** every task is a GitHub issue. User stories use the template: *As a [user], I can [action] so that [benefit]*, followed by acceptance criteria.
- **Labels:** `story`, `bug`, `tech-debt`, `docs`, and one label per service.
- **Milestones:** one per phase (Phase 1 to Phase 7).
- **Sprint rhythm:** plan on day 1, review progress mid-week, and write a short retrospective at the end (what went well, what to change).

2.7 Documentation structure

```
docs/
  00-project-foundation.md
  01-requirements.md
  02-architecture.md
  03-detailed-design.md
  04-test-plan.md
  05-deployment.md
  adr/
    0001-use-rabbitmq.md
    0002-token-bucket-rate-limiter.md
    ...
  api/                OpenAPI specs
  retrospectives/
```

The README at the repo root links to all of these.

2.8 Definition of Ready and Definition of Done

A story is Ready when: it has a clear description, acceptance criteria, a rough size, and no unresolved dependency.

A story is Done when:

- Code is merged to `main` through a reviewed pull request
- Unit tests are written and passing
- Integration tests cover the main path where relevant
- CI is green
- API changes are reflected in the OpenAPI spec
- Docs or README are updated if behavior changed
- No known critical bug remains

2.9 Timeline

A realistic plan for one person, about 4 to 5 weeks at part-time pace. Adjust to your availability.

Week	Focus
Week 0	Phase 0 and Phase 1: this document, repo setup, board, requirements sign-off
Week 1	Phase 2 and Phase 3: architecture, ADRs, database schema, API contracts
Weeks 2 to 3	Phase 4 sprints: gateway, auth, ingestion, broker, analysis
Week 4	Phase 4 continued: AI insight service, caching, React dashboard
Week 5	Phase 5 to 7: load tests, CI/CD polish, monitoring, README and demo

2.10 Phase 0 exit checklist

- Repository created with the folder structure and branch protection on `main`
- GitHub Projects board and labels created
- Development environment checklist complete
- `.gitignore` and `.env.example` committed
- This document saved as `docs/00-project-foundation.md`
- Sprint 1 planned

3. Phase 1: Planning and requirements

3.1 Problem statement

Small teams run several services and collect a flood of logs and alerts. When something breaks, engineers search through raw logs to figure out what happened, how serious it is, and where to start. This is slow and stressful, and the same errors get investigated again and again.

Pulse reduces that effort. It collects logs and metrics, notices abnormal error rates, groups related errors into a single incident, and produces a plain-English summary and a likely cause using an LLM, while avoiding repeated AI calls for errors it has already seen.

3.2 Stakeholders and personas

Persona	Description	Main needs
Developer	Builds and runs one or more services	Send logs easily, see incidents quickly, understand them fast
Admin	Manages the Pulse installation	Manage users, services, API keys and rate limits
Project owner	The author (Shivaraj)	Deliver a complete, well-documented, measurable project

3.3 Scope

In scope (MVP)

1. User registration and login with JWT
2. Service registration with API key generation
3. Ingestion API for logs and metrics
4. Spike detection based on error counts in a time window
5. Incident creation and grouping
6. AI summary and likely cause for each incident
7. React dashboard: service health, incident list, incident detail
8. Per-API-key rate limiting
9. Notification when a new incident opens (email or webhook)

Out of scope for the MVP

- Full-text log search
- Billing and multi-tenant organizations
- Mobile application
- Machine-learning anomaly detection
- Long-term log archival

3.4 User stories and acceptance criteria

US-1: Register and log in As a developer, I can create an account and log in so that I can manage my services.

- Registration validates email format and password strength.
- Passwords are stored hashed.
- Login returns a JWT with an expiry time.
- Protected endpoints reject missing or expired tokens.

US-2: Register a service and get an API key As a developer, I can register a service and receive an API key so that the service can send logs.

- The API key is shown once in full, and only a hash is stored.
- I can list my services, and revoke or regenerate a key.
- A revoked key is rejected by the gateway.

US-3: Send logs and metrics As a developer, I can send logs and metrics to an ingestion endpoint so that Pulse can analyze them.

- The endpoint accepts a batch of events with timestamp, level, message and service.
- Invalid payloads return a clear `400` error.
- Accepted events return quickly (asynchronous processing).
- Re-sending the same event does not create duplicates.

US-4: Rate limiting As the platform, I limit requests per API key so that no client can overload the system.

- Requests over the limit receive `429` with a `Retry-After` header.
- The limit is configurable per key.
- The limiter stays correct when several gateway instances run at once.

US-5: Detect error spikes As a developer, I want error spikes detected automatically so that I find out about problems without watching logs.

- A spike is detected when errors for a service in a window exceed a configured threshold.
- Threshold and window are configurable per service.
- Detection does not trigger repeatedly for an already-open incident.

US-6: Incidents As a developer, I see an incident when a spike occurs so that I can track and resolve it.

- An incident has a status (open, acknowledged, resolved), start time, affected service and a sample of related errors.
- Related errors are grouped using a normalized error pattern.
- I can change an incident's status.

US-7: AI summary As a developer, I read an AI-generated summary and likely cause for each incident so that I understand it faster.

- The summary is generated from a sample of grouped errors.
- If the AI service is slow or down, the incident still appears with a fallback message and the summary is retried later.
- Identical error patterns reuse a cached summary instead of calling the LLM again.

US-8: Dashboard As a developer, I use a dashboard to see service health and incidents at a glance.

- The overview shows each service's status and recent error rate.
- The incident list can be filtered by service and status.
- The incident page shows the timeline, sample errors and the AI summary.

US-9: Notifications As a developer, I am notified when a new incident opens so that I can react quickly.

- A notification is sent once per new incident, not once per error.
- Notification failure does not block incident creation.

3.5 Functional requirements

ID	Requirement	Related story
FR-1	The system shall allow users to register and authenticate using JWT	US-1
FR-2	The system shall let users register services and issue hashed, revocable API keys	US-2
FR-3	The gateway shall authenticate ingestion requests by API key	US-2, US-3
FR-4	The ingestion API shall accept batches of log and metric events and process them asynchronously	US-3
FR-5	The system shall enforce a configurable token-bucket rate limit per API key	US-4
FR-6	The system shall detect error spikes per service using a configurable threshold and window	US-5
FR-7	The system shall create, group and track incidents with status	US-6
FR-8	The system shall generate an AI summary and likely cause for each incident	US-7
FR-9	The system shall cache AI summaries by normalized error pattern	US-7
FR-10	The system shall cache frequently requested dashboard queries	US-8
FR-11	The dashboard shall show service health, incident list and incident detail	US-8
FR-12	The system shall notify users when a new incident opens	US-9

3.6 Non-functional requirements

All numbers are targets to be validated by load testing.

Category	Requirement
Performance	Ingestion endpoint sustains 500 requests per second *(target)* with p95 latency under 200 ms *(target)*
Cache effectiveness	AI summary cache hit rate above 60% for repeated error patterns *(target)*
Availability	Ingestion keeps accepting events when the AI service or notification service is down
Resilience	The AI call has a timeout, retry with backoff and a circuit breaker
Scalability	Services are stateless so more instances can be added; shared state lives in Redis, MySQL and the broker

Security	Hashed passwords and API keys, JWT expiry, input validation, secrets kept out of source control, no sensitive data sent to the LLM without scrubbing
Observability	Every service exposes health and metrics endpoints; request IDs are propagated across services
Maintainability	Automated tests run in CI; code follows a consistent style; schema changes go through migrations
Portability	The whole system starts with a single Docker Compose command
Cost control	LLM calls are minimized through caching and a per-incident call limit

3.7 Assumptions and constraints

Assumptions

- Client services can send HTTP requests with an API key.
- A single-region, single-node deployment is enough for the MVP.
- Log volume stays within what one broker and one database can handle for a demo.

Constraints

- One developer, part-time schedule.
- LLM usage is limited by API cost and rate limits.
- Free or low-cost hosting only.

3.8 Risks and mitigations

Risk	Likelihood	Impact	Mitigation
Scope creep	High	High	Stick to the MVP list; log new ideas in the backlog
LLM latency or cost	Medium	Medium	Caching, timeouts, a per-incident call limit, fallback message
Rate limiter bugs under concurrency	Medium	High	Atomic Lua script in Redis and concurrency tests
Too many services for one developer	Medium	Medium	Start with the gateway, auth, ingestion and analysis; add the notification service last
Sensitive data in logs reaching the LLM	Medium	High	Scrub emails, tokens and IDs before building prompts
Unrealistic performance targets	Medium	Low	Treat targets as hypotheses and report measured results honestly

3.9 Glossary

Term	Meaning
Incident	A group of related errors that crossed a detection threshold
Spike	An error count above the threshold within a time window
Token bucket	A rate-limiting algorithm where requests spend tokens that refill at a steady rate
ADR	Architecture Decision Record: a short note explaining one design decision
p95 latency	The response time that 95% of requests are faster than

Idempotent	Safe to repeat without changing the result

3.10 Phase 1 exit checklist

- Problem statement, scope and personas reviewed
- All user stories created as GitHub issues with acceptance criteria
- Functional and non-functional requirements saved as `docs/01-requirements.md`
- Risks recorded and revisited
- Stories prioritized and assigned to sprints
- Requirements frozen for the MVP (changes now go through the backlog)

4. What comes next

Phase 2: System design (HLD) covers the architecture diagram, service boundaries, data flow, technology ADRs and the deployment view. **Phase 3: Detailed design (LLD)** then covers the MySQL schema, OpenAPI contracts, event schemas, and the rate limiter and cache designs.